

Recap Multi-Agent Systems

Prolog

1.Prolog Queries

= Match

\= Mismatch

== Equals (left and right evaluation)

=\= Inequality

<, >, <=, >=

is Evaluate right side, matches with the left

?- $X = f(a)$.

?- $X = f(a)$.

$X = f(a)$.

$$?- f(X, g(a)) = f(b, g(Z)).$$

?- $f(X, g(a)) = f(b, g(Z))$.

$X = b, Z = a$.

?- $f(X,a) = f(b,X)$.

?- $f(X,a) = f(b,X)$.
false.

?- $X + 5$ is 3.

?- $X + 5$ is 3.

false.

?- $X + 5$ is $3 + 5$.

?- $X + 5$ is $3 + 5$.

false.

2. Prolog Search

Three Principles

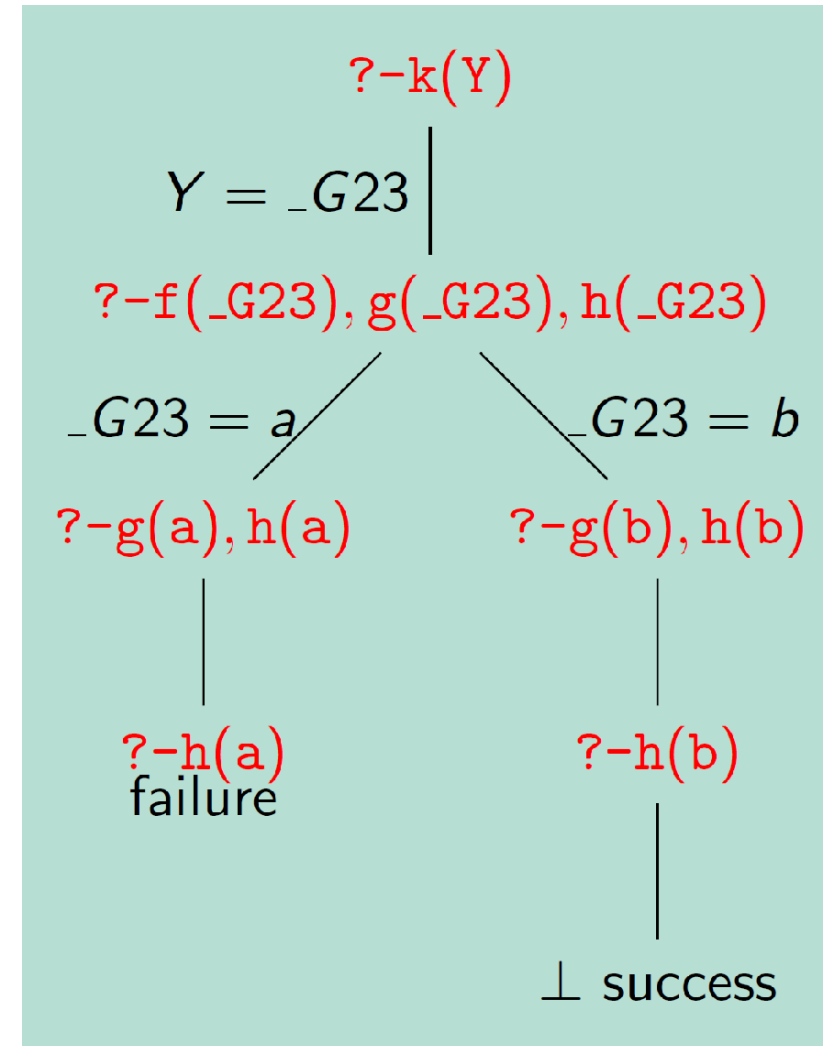
1. **backward chaining:** start from the current goal and select the rule/fact which allows to derive the goal. If a rule is matched, try to derive the rule's body (from left to right).
1. **linear:** traverse the logic program from top to bottom (select appropriate rules).
1. **backtracking:** if the current goal cannot further be matched with any rule, try other alternative rules in step 2 (i.e., the 2nd topmost, 3rd topmost, etc.). Backtrack to the closest possible choice point and try an alternative.

Search tree example

```
f(a).
f(b).
g(a).
g(b).
h(b).
k(X) :- f(X), g(X), h(X).
```

```
?- k(Y)
```

Use 'trace' and read the output.



3. Prolog Concepts

Concepts (not exhaustive)

- Atom, term, compound term, etc.
- Reachable
- Backchaining
- Backtracking
- ...

Prolog Programming

Programming a predicate

1. What does the predicate do?
2. What tools do I need?
 - a. recursion
 - b. matching
 - c. arithmetic
 - d. list notations
 - e. anonymous variable

Programming: Recursion

1. Base clause
 - a. Simple, yet as broad as possible
 - b. Adapt for procedural fit with recursive clause
 - c. e.g. descendant(X,Y):-child(X,Y).
2. Recursive clause.
 - a. Is logically correct (declarative meaning).
 - b. Reduces the 'size' of the problem (moves towards base clause)
 - c. Adapt for procedural fit with recursive clause
 - d. e.g. descendant(X,Y):- child(X,Z), descendant(Z,Y).

Example (Length of a list)

Define a predicate `length/2` which computes the length of a list:

`% base case`

`length([], 0).`

`% recursive clause`

`length([H|Tail], N) :- length(Tail, P), N is P + 1.`